编译原理
3. 语法分析-自顶向下

rainoftime.github.io
浙江大学
计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
3. **Parsing**
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

本讲内容

1 递归下降分析概述

2 LL(1)和预测分析法

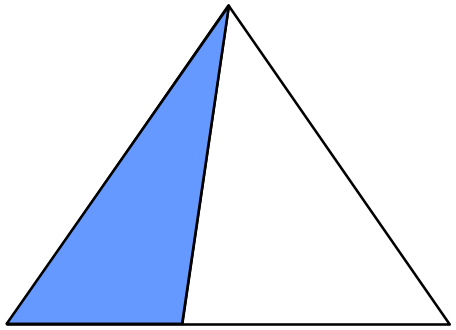
3 文法改造

4 错误恢复

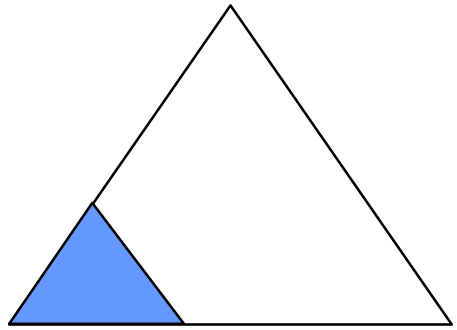
1. 递归下降分析概述

回顾: 语法分析的主要方法

- **自顶向下(Top-down)**
  - 从文法的开始符号S出发，尝试推导 (derive)出输入串
  - 分析树的构造方法：从根部开始
- **自底向上(Bottom-up)**
  - 尝试根据产生式规则归约(reduce)到文法的开始符号S
  - 分析树的构造方法: 从叶子开始



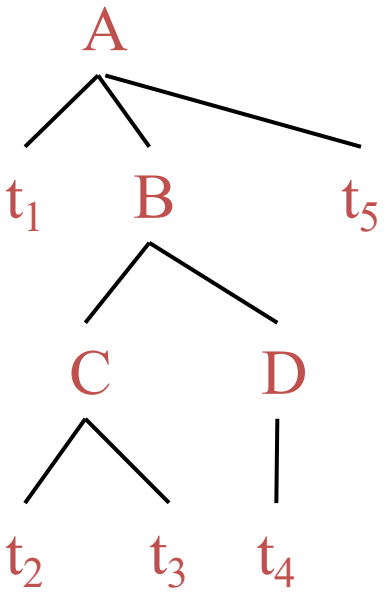
Top-down



Bottom-up

自顶向下语法分析

- **从文法开始符号*S*推导出串*w***
  - **E.g.,** t_1 t_2 t_3 t_4 t_5
  - 从顶部向底部方向构造Parse Tree
    - 从上往下、从左往右



- **每一步推导中，都需要做两个选择**
  1. 替换当前句型中的哪个非终结符?
  2. 用该非终结符的哪个产生式进行替换

自顶向下语法分析

- **从分析树的顶部向底部方向构造分析树**
  - 从文法开始符号S推导出串w

- **每一步推导中，都需要做两个选择**
  1. 替换当前句型中的哪个非终结符?

自顶向下分析总是选择每个句型的最左非终结符进行替换!

2. 用该非终结符的哪个产生式进行替换?

文法 $S \rightarrow cAd$
$A \rightarrow ab \mid b$

A有2个可选的产生式

例: 自顶向下分析过程

$S \rightarrow E + S \mid E$

$E \rightarrow \mathbf{num} \mid (\ S\ )$

Construct a leftmost derivation of string while reading in token stream

Partly-derived String	Lookahead	Parsed part Unparsed part
S	(	(1+2+(3+4))+5
→ ***E***+S	(	(1+2+(3+4))+5
→ (***S***) +S	1	(1+2+(3+4))+5
→ (***E***+S)+S	1	(1+2+(3+4))+5
→ (1+***S***)+S	2	(1+2+(3+4))+5
→ (1+***E***+S)+S	2	(1+2+(3+4))+5
→ (1+2+***S***)+S	2	(1+2+(3+4))+5
→ (1+2+***E***)+S	(	(1+2+(3+4))+5
→ (1+2+(***S***))+S	3	(1+2+(3+4))+5
→ (1+2+(***E***+S))+S	3	(1+2+(3+4))+5

(We will talk about "lookahead" later)

递归下降分析：自顶向下分析的通用形式

- **递归下降分析(Recursive-Descent Parsing):**
  - 由一组过程/函数组成，每个过程对应一个非终结符
  - 从开始符号*S*对应的过程开始，(递归)调用其它过程
  - 如果*S*对应的过程恰好扫描了整个输入串，则完成分析

文法 $S \rightarrow cAd$

$A \rightarrow ab \mid b$

如何为S和A构造其对应的过程?

递归下降分析:自顶向下分析的通用形式

- **递归下降分析(Recursive-Descent Parsing):**
  - 利用非终结符A的规则$A \rightarrow X_1 \ldots X_k$，定义识别$A$的过程
    - **如果X_i是非终结符**：调用相应非终结符对应的过程
    - **如果X_i是终结符a**：匹配输入串中对应的终结符a

```
    void A( ) {
1)    选择一个A产生式，A → X1 X2 ... Xk ;
2)    for ( i = 1 to k ) {
3)      if ( Xi是一个非终结符号)
4)          调用过程 Xi( ) ;
5)      else  if ( Xi 等于当前的输入符号a)
6)          读入下一个输入符号;
7)      else /* 发生了一个错误 */ ;
8)            ？？？
      }
    }
```

如果选择了不合适的产生式，可能需要**回溯**

注: 最后我们会给相应的代码实现例子

例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$建立分析树(假设按顺序选产生式)

cad ***S***

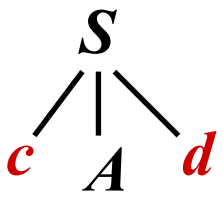
例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$ 建立分析树(假设按顺序选产生式)

cad



- **选择S的产生式(只有1个选择)**

例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$ 建立分析树(假设按顺序选产生式)

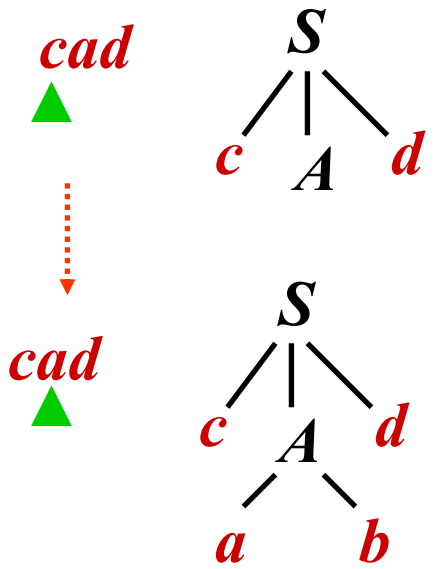
- **选择S的产生式(只有1个选择)**

例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$ 建立分析树(假设按顺序选产生式)



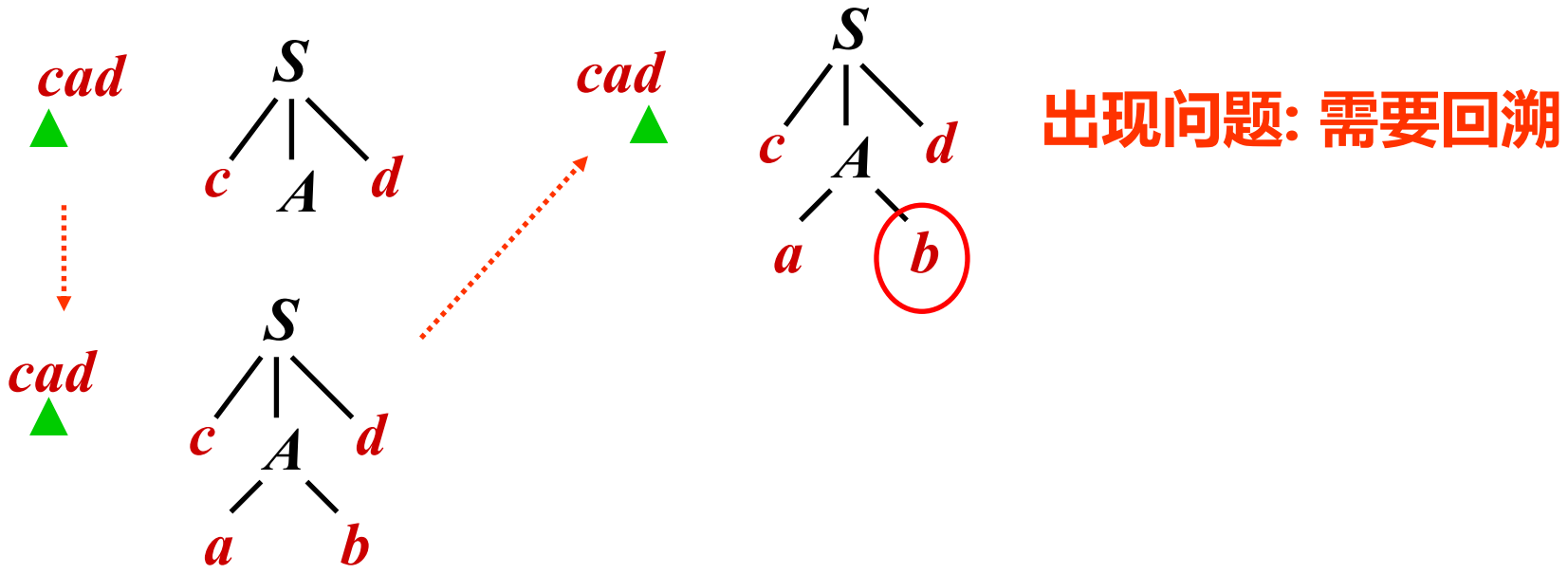
- **选择A的第1个产生式**$A \rightarrow ab$

例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$ 建立分析树(假设按顺序选产生式)



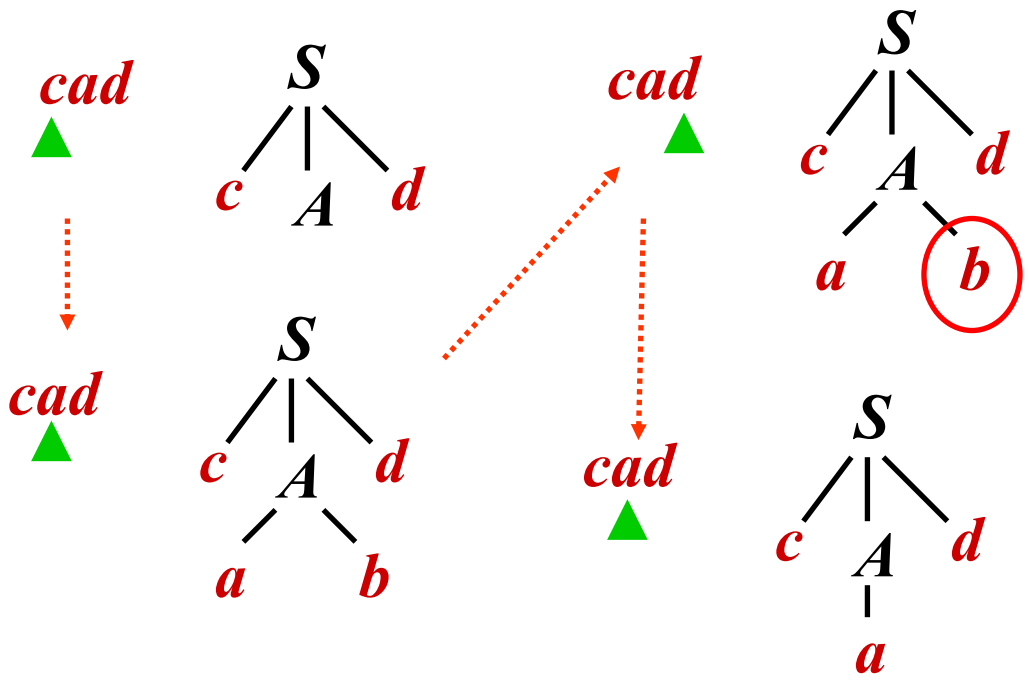
出现问题: 需要回溯

例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$ 建立分析树(假设按顺序选产生式)



出现问题: 需要回溯

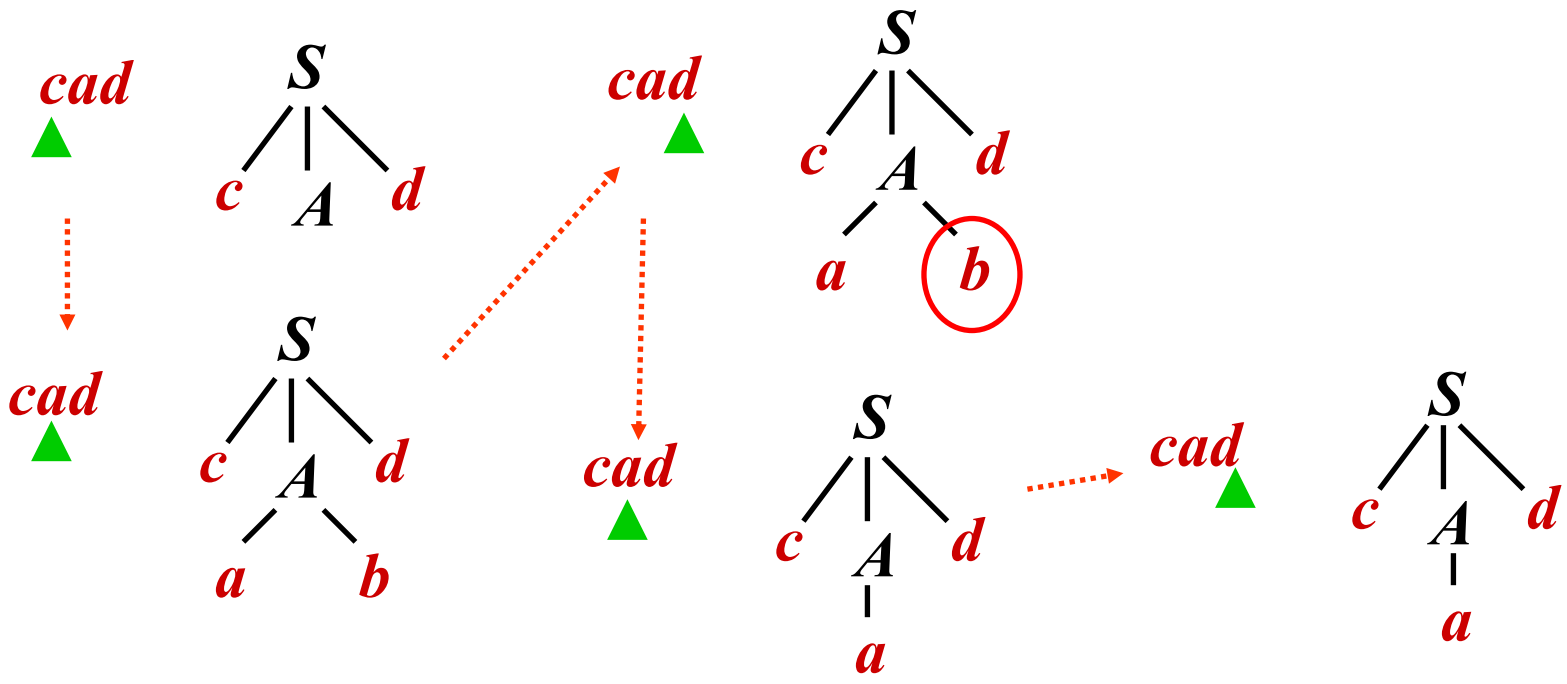
- **回到上一步, 重新选择A的第2个产生式$A \rightarrow a$**

例: 需要回溯的文法

考虑文法$S \rightarrow cAd$

$A \rightarrow ab \mid a$

为输入串$w = cad$ 建立分析树(假设按顺序选产生式)



- **成功匹配了输入串cad**

通用递归下降的问题: 回溯

- **复杂的回溯→代价太高**
  - 非终结符有可能有多个产生式，由于信息缺失，无法准确预测选择哪一个
  - 考虑到往往需要对多个非终结符进行推导展开，因此尝试的路径可能呈指数级爆炸

    例如：$A \rightarrow ab \mid a$，不知该选择哪条产生式
- **其分析过程类似于NFA**

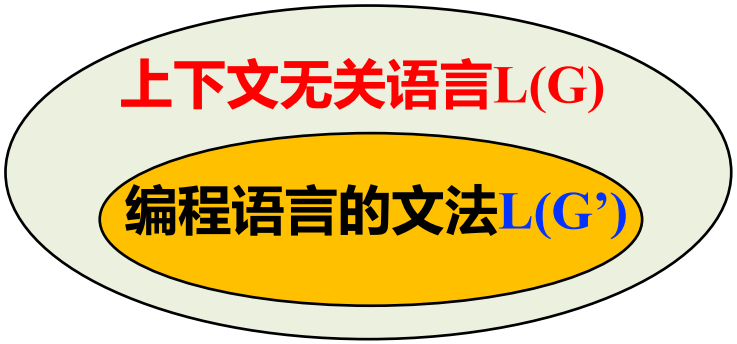
是否可以构造一个类似于DFA的分析方法？

回顾: 语法分析的复杂度

对于一个终结符号串*x*, 从*S*推导出*x*或者将*x*归约为*S*

- **If there are no restrictions on the form of grammar used, parsing CFL requires $O(n^3)$ time**
  - E.g., Cocke-Younger-Kasami's algorithm (CYK)

- **Subsets of CFLs typically require O(n) time**
  - Predictive parsing using **LL(1)** grammars
  - Shift-Reduce parsing using **LR(1)** grammars



为了实现高效搜索, 控制搜索空间, 如文法产生式的限制

预测分析法(Predictive Parsing)

此方法接受LL(k)文法!

- L: “left-to-right” 从左到右扫描
- L: “leftmost derivation” 最左推导
- k: 向前看k个Token来确定产生式 (通常k=1)
  - 每次为**最左边的非终结符号**选择产生式时，向前看1个输入符号，预测要使用的产生式

文法加什么限制可以保证没有回溯？

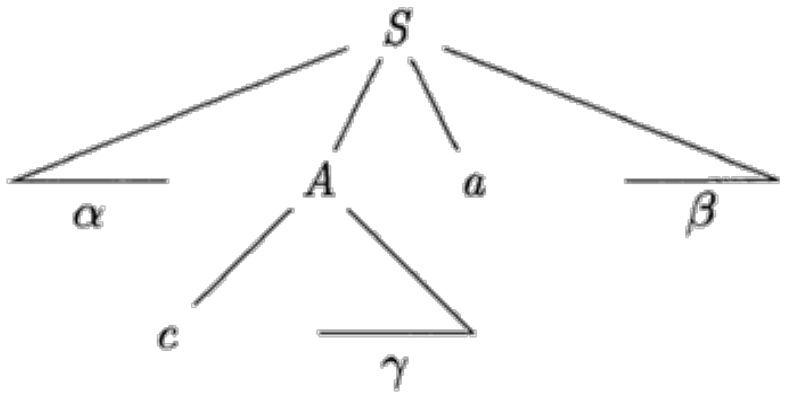
2. LL(1)和预测分析法

- LL(1)文法
  - First, Follow集
  - LL(1)文法定义
- 实现LL(1)预测分析

First集和Follow集

给定 $G = (T, N, P, S)$, $\alpha \in (T \cup N)^*$

- **First(α)** = $\{a \mid \alpha \Rightarrow^* a\ldots, a \in T\}$
  - 可从α推导得到的串的首个终结符的集合

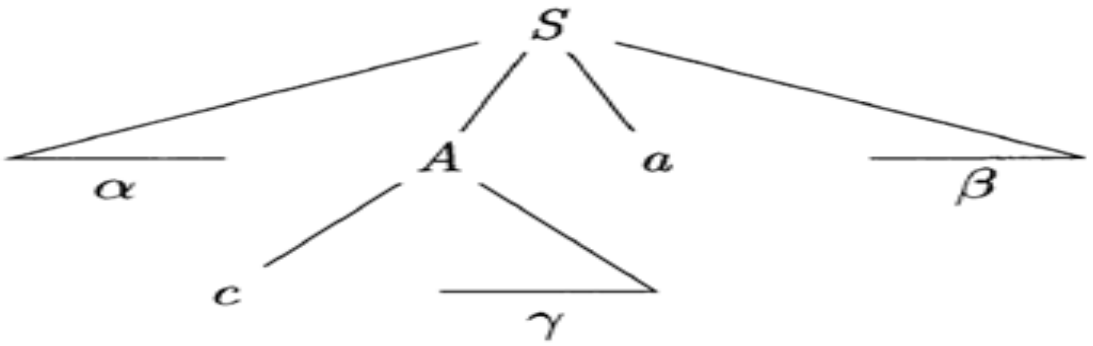


终结符号 c 在 FIRST(A)

First集和Follow集

给定 $G = (T, N, P, S)$，$A \in N$

- **Follow(A)** = $\{a \mid S \Rightarrow^* \ldots Aa \ldots ，a \in T\}$
  - 从S出发, 可能在推导过程中跟在A右边的终结符号集
  - 例如:$S \rightarrow \alpha A a \beta$，终结符号a ∈ Follow($A$)



a 在 FOLLOW(A)中

小结： First集和Follow集

给定 $G = (T, N, P, S)$, $\alpha \in (T \cup N)^*$, $A \in N$

- **First(α)** = $\{a \mid \alpha \Rightarrow^* a\ldots, a \in T\}$
  - 可从α推导得到的串的首个终结符的集合

- **Follow(*A*)** = $\{a \mid S \Rightarrow^* \ldots Aa\ldots\ ,\ a \in T\}$
  - 从S出发, 可能在推导过程中跟在A右边的终结符号集

- First(α): for arbitrary string of terminals and non-terminals α
- Follow(X) for a non-terminal X

Nullable集的归纳定义

- First, Follow集涉及**空串**，我们引入Nullable概念

- If X can derive an empty string (**X is Nullable**), **iff**
  - **Base case**:
    - $X \rightarrow \varepsilon$: then X must be Nullable
  - **Inductive case**:
    - $X \rightarrow Y_1 \ldots Y_n$

      If $Y_1, \ldots, Y_n$ are n nonterminals and **may all derive** empty strings, then X is Nullable.

根据归纳定义计算Nullable集

- Compute the Nullable set by iteration:

```
/* Nullable: a set of nonterminals */
Nullable <- {};
while (Nullable still changes)
  for (each production X → α)
    switch (α)
    case ε:
      Nullable U= {X};
      break;
    case Y1 … Yn:
      if (Y1∈Nullable && … && Yn∈Nullable)
        Nullable U= {X};
      break;
```

First集的归纳定义

$$G = (T, N, P, S),\ \alpha \in (T \cup N)^*,\ A \in N$$

- **First(α) = {a | α ⇒* a…, $a \in T$ }**
  可从α推导得到的串的首终结符号集合
  - **Base case:**
    - If X is a terminal: First (X) = {X}
  - **Inductive case:**
    - If $X \rightarrow Y_1\ Y_2\ \ldots\ Y_n$
      - First (X) ∪= First(Y_1)
      - If $Y_1 \in$ Nullable, First (X) ∪= First(Y_2)
      - If $Y_1, Y_2 \in$ Nullable, First (X) ∪= First(Y_3)
      - ……

等价于First (X) ⊇ First(Y_1)

注: 上述规则看起来似乎是关于非终结符的。但是First是关于文法符号串α（如产生式右部) 的，规则如inductive case

Follow集的归纳定义

$$G = (T, N, P, S),\ \alpha \in (T \cup N)^*,\ A \in N$$

- **Follow(A)** = $\{a \mid S \Rightarrow^* \ldots Aa \ldots ,\ a \in T\}$
  从S出发, 可能在推导过程中跟在A右边的终结符号集
  - **Base case:**
    - Follow (A) = {}
  - **Inductive case:**
    - If B → s1 A s2 for any s1 and s2
      1. Follow(A) ∪= First(s2)
      2. If s2 is Nullable, Follow(A) ∪= Follow(B)

关于第2种情况, 假设S =>* …B b… (b属于Follow(B))
- 用s1 A s2替换B后：S =>* …s1 A s2 b …
- 由于s2 is Nullable, 因此b也属于Follow(A)!

Follow集的归纳定义

- Follow Set的归纳定义 B -> s1 A s2是一般情况，对于一些特殊的产生式，可能可以运用简单的推论，比如

Production	Constraints
$T' \rightarrow T\$$	$\{\$\} \subseteq FOLLOW(T)$
$T \rightarrow R$	$FOLLOW(T) \subseteq FOLLOW(R)$
$T \rightarrow \mathtt{a}T\mathtt{c}$	$\{\mathtt{c}\} \subseteq FOLLOW(T)$
$R \rightarrow \varepsilon$	
$R \rightarrow R\mathtt{b}R$	$\{\mathtt{b}\} \subseteq FOLLOW(R)$

Base case:

Follow (A) = {}

Inductive case:

If B → s1 A s2 for any s1 and s2

1. Follow(A) ∪= First(s2)
2. If s2 is Nullable, Follow(A) ∪= Follow(B)

例: 计算Nullable, First和Follow集

$Z \rightarrow d$	$Y \rightarrow c$	$X \rightarrow Y$
$Z \rightarrow X\ Y\ Z$	$Y \rightarrow \varepsilon$	$X \rightarrow a$

注意：有的地方(如虎书) 写作

$Y \rightarrow c$

$Y \rightarrow$

第2个产生式"$Y \rightarrow$ "也表示" $Y \rightarrow \varepsilon$"

To know the possible first terminal symbols of each production, we need to calculate the Nullable, First, and Follow Sets

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

- 初始化为False(或者说Nullable集合为空)

	Nullable	First	Follow
Z	False		
Y	False		
X	False		

例: 计算Nullable, First, Follow集

$Z \to d$
$Z \to X\ Y\ Z$

$Y \to c$
$Y \to \varepsilon$

$X \to Y$
$X \to a$

- **Since $Y \to \varepsilon$, Nullable ∪= {Y}; (Base Case)**

	Nullable	First	Follow
Z	False		
Y	True		
X	False		

注: Nullable ∪= {Y}等价于Nullable = Nullable ∪ {Y}

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

- **Since $X \rightarrow Y$ and Y is Nullalbe, Nullable ∪= {X} (Inductive Case)**

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

- **Stop the iteration here**

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

- **Initialize the First sets**

	Nullable	First	Follow
Z	False	{}	
Y	True	{}	
X	True	{}	

例: 计算Nullable, First, Follow集

$Z \to d$
$Z \to X\ Y\ Z$

$Y \to c$
$Y \to \varepsilon$

$X \to Y$
$X \to a$

	Nullable	First	Follow
Z	False	{d}	
Y	True	{c}	
X	True	{a}	

Base case: If X is a terminal then we have First (X) = {X}

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

	Nullable	First	Follow
Z	False	{d}	
Y	True	{c}	
X	True	{a, c}	

Inductive Case

If X is a terminal: First (X) = {X}

If X -> $Y_1\ Y_2 \ldots Y_n$

- First (X) ∪= First(Y_1)
- If $Y_1 \in$ Nullable, First (X) ∪= First(Y_2)
- If $Y_1, Y_2 \in$ Nullable, First (X) ∪= First(Y_3)
- ……

例: 计算Nullable, First, Follow集

$Z \to d$
$Z \to X\ Y\ Z$

$Y \to c$
$Y \to \varepsilon$

$X \to Y$
$X \to a$

	Nullable	First	Follow
Z	False	{d, a, c}	
Y	True	{c}	
X	True	{a, c}	

If X is a terminal: First (X) = {X}

If X -> $Y_1\ Y_2 \ldots Y_n$

- First (X) ∪= First(Y_1)
- If $Y_1 \in$ Nullable, First (X) ∪= First(Y_2)
- If $Y_1, Y_2 \in$ Nullable, First (X) ∪= First(Y_3)
- ……

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

	Nullable	First	Follow
Z	False	{d, a, c}	
Y	True	{c}	
X	True	{a, c}	

Iterate again.
No change.
Stop iteration.

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

- **Initialize the follow sets as {}**

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{}
X	True	{a, c}	{}

例: 计算Nullable, First, Follow集

$Z \to d$
$Z \to X\ Y\ Z$

$Y \to c$
$Y \to \varepsilon$

$X \to Y$
$X \to a$

- **利用Inductive Case的情况1: Follow(Y) ∪= First(Z)**

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{}

B → s1 A s2 for any s1, s2

1. **Follow(A) ∪= First(s2)**
2. If s2 is Nullable:
   Follow(A) ∪= Follow(B)

例: 计算Nullable, First, Follow集

$Z \to d$
$Z \to X\ Y\ Z$

$Y \to c$
$Y \to \varepsilon$

$X \to Y$
$X \to a$

- **利用Inductive Case的情况1: Follow(X) ∪= First(YZ) = ?**

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

$B \to s1\ A\ s2$ for any s1, s2

1. **Follow(A) ∪= First(s2)**
2. If s2 is Nullable:
   Follow(A) ∪= Follow(B)

s1视作空串，则s2对应YZ。
因此Follow(X) ⊇ First(YZ)

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

B → s1 A s2 for any s1, s2

1. Follow(A) ∪= First(s2)
2. If s2 is Nullable:
   Follow(A) ∪= Follow(B)

例: 计算Nullable, First, Follow集

$Z \to d$
$Z \to X\ Y\ Z$

$Y \to c$
$Y \to \varepsilon$

$X \to Y$
$X \to a$

- **利用Inductive Case的情况2: Follow(Y) ∪= Follow(X)**

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

B → s1 A s2 for any s1, s2

1. Follow(A) ∪= First(s2)
2. If s2 is Nullable:
   Follow(A) ∪= Follow(B)

例: 计算Nullable, First, Follow集

$Z \rightarrow d$
$Z \rightarrow X\ Y\ Z$

$Y \rightarrow c$
$Y \rightarrow \varepsilon$

$X \rightarrow Y$
$X \rightarrow a$

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

优化Nullable, First, Follow的计算

- 上述过程按顺序算了Nullable, First, Follow集
- 实际上，它们可以同时计算! See Tiger book algorithm 3.13

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$
        for each i from 1 to k, each j from $i+1$ to k,
            if all the Y_i are nullable
                then nullable[X] $\leftarrow$ true
            if $Y_1 \cdots Y_{i-1}$ are all nullable
                then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
            if $Y_{i+1} \cdots Y_k$ are all nullable
                then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
            if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
                then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

不要求必需掌握，选适合的即可😄
(有人可能觉得Alg. 3.13更好用)

优化Nullable, First, Follow的计算

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$ for each production
        for each i from 1 to k, each j from $i+1$ to k,
            if all the Y_i are nullable
                then nullable[X] $\leftarrow$ true
            if $Y_1 \cdots Y_{i-1}$ are all nullable
                then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
            if $Y_{i+1} \cdots Y_k$ are all nullable
                then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
            if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
                then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

优化Nullable, First, Follow的计算

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$
      for each i from 1 to k, each j from $i+1$ to k,
        if all the Y_i are nullable
          then nullable[X] $\leftarrow$ true
        if $Y_1 \cdots Y_{i-1}$ are all nullable
          then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
        if $Y_{i+1} \cdots Y_k$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
        if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

compute nullable iteratively

优化Nullable, First, Follow的计算

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$
      for each i from 1 to k, each j from $i+1$ to k,
        if all the Y_i are nullable
          then nullable[X] $\leftarrow$ true
        if $Y_1 \cdots Y_{i-1}$ are all nullable
          then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
        if $Y_{i+1} \cdots Y_k$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
        if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

Analyze each **prefix** Y1…Yi-1

优化Nullable, First, Follow的计算

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$
      for each i from 1 to k, each j from $i+1$ to k,
        if all the Y_i are nullable
          then nullable[X] $\leftarrow$ true
        if $Y_1 \cdots Y_{i-1}$ are all nullable Analyze each **suffix** Yi+1…Yk
          then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
        if $Y_{i+1} \cdots Y_k$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
        if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

优化Nullable, First, Follow的计算

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$
      for each i from 1 to k, each j from $i+1$ to k,
        if all the Y_i are nullable
          then nullable[X] $\leftarrow$ true
        if $Y_1 \cdots Y_{i-1}$ are all nullable
          then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
        if $Y_{i+1} \cdots Y_k$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
        if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

X -> Y1 Y2 ... Yi ..... Yj ... Yk

this substring

Analyze each **substring**,
Yi+1...Yj-1 can be **empty**

优化Nullable, First, Follow的计算

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.
for each terminal symbol Z
    FIRST[Z] $\leftarrow$ {Z}
repeat
    for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$
      for each i from 1 to k, each j from $i+1$ to k,
        if all the Y_i are nullable
          then nullable[X] $\leftarrow$ true
        if $Y_1 \cdots Y_{i-1}$ are all nullable
          then FIRST[X] $\leftarrow$ FIRST[X] $\cup$ FIRST[Y_i]
        if $Y_{i+1} \cdots Y_k$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FOLLOW[X]
        if $Y_{i+1} \cdots Y_{j-1}$ are all nullable
          then FOLLOW[Y_i] $\leftarrow$ FOLLOW[Y_i] $\cup$ FIRST[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

Iteration

2. LL(1)和预测分析法

- LL(1)文法
  - First, Follow集
  - LL(1)文法的定义
- 实现LL(1)预测分析

LL(1)文法

- 预测分析器（自顶向下）在展开非终结符 A 时，只看*当前一个输入符号*就必须唯一确定选哪条产生式。

$$A \rightarrow \alpha \mid \beta$$

- 那么，文法本身需要满足什么条件，才能保证这种"唯一性"？

LL(1)文法的定义

- 文法G的**任何两个**产生式$A \to \alpha \mid \beta$ 都满足下列条件：

  1. **First(α) ∩ First(β) = ∅**
     - α和β推导不出以同一个单词为首的串

  2. **若β ⇒* ε ，那么α ⇏* ε , 且First(α) ∩ Follow(*A*) = ∅**
     - α和β不能同时推出ε;
     - First(α)不应在Follow(A) 中

以上条件可以保证产生式选择的唯一性

LL(1)文法的定义: 条件 1

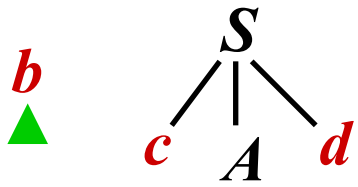
- 文法G的**任何两个**产生式$A \to \alpha \mid \beta$ 都满足下列条件：

1. $First(\alpha) \cap First(\beta) = \varnothing$

α和β推导不出以同一个单词为首的串

意义: 假设下一个输入是***b***，且First(α)和First(β)不相交。

- 若$b \in First(\alpha)$，则选择$A \to \alpha$；
- 若$b \in First(\beta)$，则选择$A \to \beta$



2. 若$\beta \Rightarrow^* \varepsilon$，那么$\alpha \not\Rightarrow^* \varepsilon$，且$First(\alpha) \cap Follow(A) = \varnothing$

α和β不能同时推出ε; First(α)不应在Follow(A) 中

LL(1)文法的定义: 条件 2

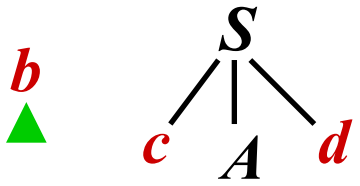
- 文法G的**任何两个**产生式$A \to \alpha \mid \beta$ 都满足下列条件：

1. First(α) ∩ First(β) = ∅

α和β推导不出以同一个单词为首的串

意义: 假设下一个输入是***b***，且First(*α*)和First(*β*)不相交。

- 若$b \in First(\alpha)$，则选择$A \to \alpha$；
- 若$b \in First(\beta)$，则选择$A \to \beta$



2. 若β ⇒* ε ，那么α ⇏* ε , 且First(α) ∩ Follow(*A*) = ∅

2.1 α和β不能同时推出ε; 2.2 First(α)不应在Follow(A) 中

意义: 假设下一个输入是***b***, 且 β =>* ε

- 如果$b \in First(\alpha)$，则选择$A \to \alpha$ *(类似上面条件1的情况)*
- 如果$b \in Follow(A)$，则选择$A \to \beta$，因为最终到达了ε且后面跟着*b*

LL(1)文法的定义: 条件 2

- **场景推演：当 $\beta \Rightarrow^* \varepsilon$，当前输入为 b**

b 的位置	应该选哪条产生式？	依据	结论
b ∈ First(α)	A → α	b 是 α 推导内容的开头	✓ 选 α
b ∈ Follow(A)	A → β （再由 β ⇒* ε 消失）	A 后面跟着 b，让 A 为空， b 属于 A 的后继	✓ 选 β
b ∈ First(α) ∩ Follow(A)	A→α 还是 A→β？	两种情况同时触发	✗ 冲突！

- **FOLLOW(A)的直觉**
  - **Follow(A)** = 在所有句型中，紧跟在 A *后面*出现的终结符集合
  - 当选择 A→β 且 β⇒*ε 时，A 相当于"不产生任何内容"，接下来读到的就是 Follow(A) 中的符号
  - 条件2.1 保证：能触发"A→α"的符号集合 与 能触发"A→β（消失）"的符号集合 没有交集

2. LL(1)和预测分析法

- LL(1)文法的定义
- 实现LL(1)预测分析
  - 构造预测分析表
  - 运用预测分析表

回顾: 自顶向下语法分析

- **从分析树的顶部向底部构造分析树**
  - 从文法开始符号S推导出串w

- **每一步推导中，都需要做两个选择**
  1. **替换当前句型中的哪个非终结符?**
     - 总是选择最左非终结符进行替换!
  2. **用该非终结符的哪个产生式进行替换?**

利用First, Follow信息辅助决策！

预测分析表

- **行**A：对应一个**非终结符**
- **列**a：对应某个**终结符或输入结束符**$
- **项M(A,a)**：针对非终结符为A，当下一个输入Token为a时，可选的产生式

	int	*	+	$
T	int Y			
E	T X			
X			+ E	ε
Y		* T	ε	ε

Consider the [E, int] entry:

"When current non-terminal is E and next input is int, use production E → T X

预测分析表的构造

$Z \to X\ Y\ Z$　　$Y \to c$　　$X \to a$

$Z \to d$　　$Y \to \varepsilon$　　$X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

对文法G的每个产生式$X \to \gamma$

- if t ∈ First(γ): enter ($X \to \gamma$) in row X, col t
- if γ is Nullable and t ∈ Follow(X): enter ($X \to \gamma$) in row X, col t

	a	c	d
Z			
Y			
X			

预测分析表的构造

$Z \to X\ Y\ Z$　　$Y \to c$　　$X \to a$

$Z \to d$　　$Y \to \varepsilon$　　$X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

对文法G的每个产生式$X \to \gamma$

- if t ∈ First(γ): enter ($X \to \gamma$) in row X, col t
- if γ is Nullable and t ∈ Follow(X): enter ($X \to \gamma$) in row X, col t

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ
Y			
X			

First(XYZ)

预测分析表的构造

$Z \to X\ Y\ Z$ $\quad Y \to c$ $\quad X \to a$

$Z \to d$ $\quad Y \to \varepsilon$ $\quad X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

对文法G的每个产生式$X \to \gamma$

- if t ∈ First(γ): enter ($X \to \gamma$) in row X, col t
- if γ is Nullable and t ∈ Follow(X): enter ($X \to \gamma$) in row X, col t

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ Z→d
Y		Y→c	
X	X→a		

预测分析表的构造

$Z \to X\ Y\ Z$　　$Y \to c$　　$X \to a$

$Z \to d$　　$Y \to \varepsilon$　　$X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

- if t ∈ First(γ): enter ($X \to \gamma$) in row X, col t
- if γ is Nullable and t ∈ Follow(X): enter ($X \to \gamma$) in row X, col t

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ Z→d
Y	Y→ε	Y→c Y→ε	Y→ε
X	X→a		

Follow(Y) = {a, c, d}

预测分析表的构造

$Z \to X\ Y\ Z$ $\quad Y \to c$ $\quad X \to a$

$Z \to d$ $\quad Y \to \varepsilon$ $\quad X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

- if t ∈ First(γ): enter ($X \to \gamma$) in row X, col t
- if γ is Nullable and t ∈ Follow(X): enter ($X \to \gamma$) in row X, col t

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ Z→d
Y	Y→ε	Y→c Y→ε	Y→ε
X	X→a X→Y	X→Y	X→Y

Follow(X) = {a, c, d}

预测分析表的构造

$Z \to X\ Y\ Z$ $\quad Y \to c$ $\quad X \to a$

$Z \to d$ $\quad Y \to \varepsilon$ $\quad X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

- if t ∈ First(γ): enter ($X \to \gamma$) in row X, col t
- if γ is Nullable and t ∈ Follow(X): enter ($X \to \gamma$) in row X, col t

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ Z→d
Y	Y→ε	Y→c Y→ε	Y→ε
X	X→a X→Y	X→Y	X→Y

预测分析表的构造

$Z \to X\ Y\ Z$ $\quad Y \to c$ $\quad X \to a$

$Z \to d$ $\quad Y \to \varepsilon$ $\quad X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ Z→d
Y	Y→ε	Y→c Y→ε	Y→ε
X	X→a X→Y	X→Y	X→Y

What are the blanks?
--> syntax errors

预测分析表的构造

$Z \to X\ Y\ Z$　　$Y \to c$　　$X \to a$

$Z \to d$　　$Y \to \varepsilon$　　$X \to Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Is it possible to put 2 grammar rules in the same box?

	a	c	d
Z	Z→XYZ	Z→XYZ	Z→XYZ Z→d
Y	Y→ε	Y→c Y→ε	Y→ε
X	X→a X→Y	X→Y	X→Y

上面的文法不是LL(1)文法！！

利用预测分析表定义LL(1)文法

- 文法G的任何两个产生式$A \to \alpha \mid \beta$ 都满足下列条件：

  1. First(α) ∩ First(β) = ∅

  α和β推导不出以同一个单词为首的串

  2. **若β ⇒* ε ，那么α ⇒* ε , 且First(α) ∩ Follow(*A*) = ∅**

  α和β不能同时推出ε; First(α)不应在Follow(A) 中

- **LL(1) 文法**:

If a predictive parsing table constructed this way contains no conflicting entries, the grammar is called LL(1).

2. LL(1)和预测分析法

- LL(1)文法的定义
- 实现LL(1)预测分析
  - 构造预测分析表
  - 利用驱动预测表
    - 非递归实现预测分析

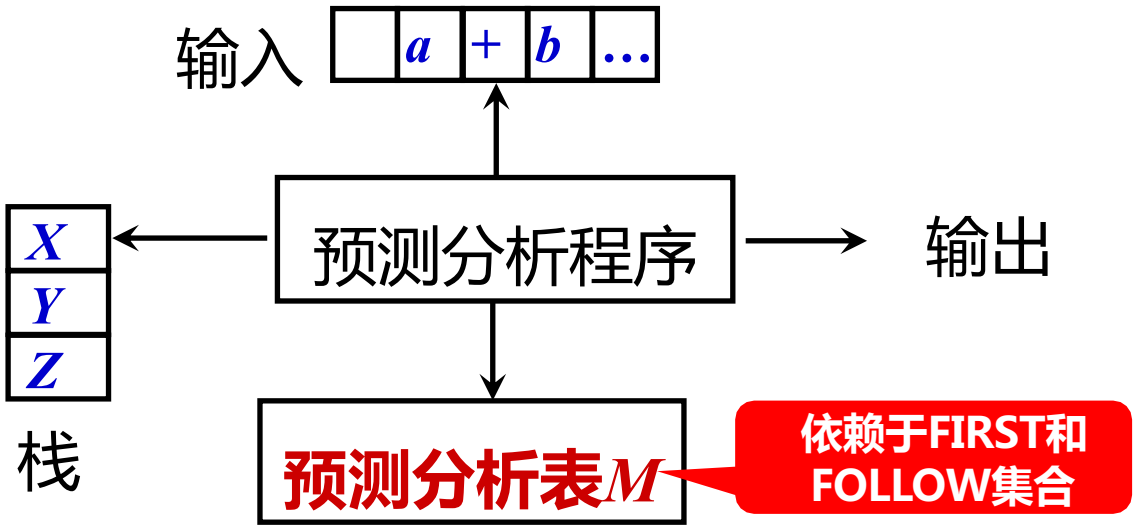
注： 一般情况不考，
也就是到构造分析表。
但是可能有助于理解
&&运用原理，
以及以防万一。。。

LL(1)语法分析的实现

- **LL(k)语法分析的含义**
  - L: “left-to-right” 从左到右扫描
  - L: “leftmost derivation” 最左推导
  - k: 向前看k个Token来确定产生式 (通常k=1)
- **LL(1)分析的非递归实现**
  - 使用显式的栈，而不是递归调用完成分析
  - 类似模拟LL文法对应的下推自动机(PDA)
- **LL(1)分析的递归实现**
  - 递归下降分析: 非终结符对应子过程

LL(1)的非递归实现

- **针对输入w的两个基本动作**
  1. 若栈顶是非终结符A：利用预测分析表, 选择产生式 $A \rightarrow \alpha$ (将栈顶的**非终结符*A*替换成串**α)
  2. 若栈顶是终结符a：栈顶记号a和输入中的Token匹配



初始状态： $w在输入缓冲区中。

(No-Recursive) Table-Driven Parsing

Stack: to keep track of what is pending in the derivation

```
stack.push($); stack.push(S);
a = input.read(); // lookahead
forever do begin
    X = stack.peek();
    if X = a and a = $ then return SUCCESS;
    elsif X = a and a != $ then
      stack.pop(X); a = input.read();
    elsif X != a and X ∈ N and M[X,a] not empty then
      stack.pop(X);
      stack.push(M[X,a]);
    else ERROR!
end
```

/* $M[X, a] = Y_1 \ldots Y_n$ */

$X \longrightarrow Y_1 \ldots Y_n$

若栈顶是非终结符X：利用预测分析表, 选择产生式$X \rightarrow \alpha$ (将栈顶的非终结符X替换成串α)

(No-Recursive) Table-Driven Parsing

Stack: to keep track of what is pending in the derivation

```
stack.push($); stack.push(S);
a = input.read(); // lookahead
forever do begin
    X = stack.peek();
    if X = a and a = $ then return SUCCESS;
    elsif X = a and a != $ then
      stack.pop(X); a = input.read();
    elsif X != a and X ∈ N and M[X,a] not empty then
      stack.pop(X);
      stack.push(M[X,a]);
    else ERROR!
end
```

/* M[X, a] = $Y_1 \ldots Y_n$ */

$X \longrightarrow Y_1 \ldots Y_n$

若栈顶是终结符a：栈顶记号a和输入中的Token匹配的话，pop掉并读入下一个lookahead符号

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	

Stack: to keep track of what is pending in the derivation

1. Terminal: Match
2. Non-terminal: derive

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S

若栈顶是非终结符X：利用预测分析表, 选择产生式$X \rightarrow \alpha$ (也就是将栈顶的**非终结符X替换成串**α)

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	

若栈顶是终结符a：栈顶记号a和输入中的Token匹配的话，pop掉并读入下一个lookahead符号

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match

若栈顶是终结符a：栈顶记号a和输入中的Token匹配的话，pop掉并读入下一个lookahead符号

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match
3	$S)S	)$	

若栈顶是非终结符X：利用预测分析表, 选择产生式$X \rightarrow \alpha$ (也就是将栈顶的**非终结符X替换成串α**)

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match
3	$S)S	)$	S -> ε

若栈顶是非终结符X：利用预测分析表, 选择产生式$X \rightarrow \alpha$ (将栈顶的**非终结符X替换成串**α)

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match
3	$S)S	)$	S -> ε
4	$S)	)$	

若栈顶是终结符a：栈顶记号a和输入中的Token匹配的话，pop掉并读入下一个lookahead符号

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match
3	$S)S	)$	S -> ε
4	$S)	)$	match

若栈顶是终结符a：栈顶记号a和输入中的Token匹配的话，pop掉并读入下一个lookahead符号

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match
3	$S)S	)$	S -> ε
4	$S)	)$	match
5	$S	$	S -> ε

若栈顶是非终结符X：利用预测分析表, 选择产生式$X \rightarrow \alpha$ (也就是将栈顶的**非终结符X替换成串**α)

Example: Table-Driven LL(1) Parsing

- S→(S) S | ε

M[N, T]	(	)	$
S	S -> (S)S	S -> ε	S -> ε

Steps	Parsing Stack	Input	Action
1	$S	()$	S -> (S)S
2	$S)S(	()$	match
3	$S)S	)$	S -> ε
4	$S)	)$	match
5	$S	$	S -> ε
6	$	$	accept

2. LL(1)和预测分析法

- LL(1)文法的定义
- 实现LL(1)预测分析
  - 构造预测分析表
  - 利用驱动预测表
    - 递归实现预测分析

注： 一般情况不考，
也就是到构造分析表。
但是可能有助于理解
&&运用原理，
以及以防万一。。。

LL(1)语法分析的实现

- **LL(k)语法分析的含义**
  - L: “left-to-right” 从左到右扫描
  - L: “leftmost derivation” 最左推导
  - k: 向前看k个Token来确定产生式 (通常k=1)
- **LL(1)分析的非递归实现**
  - 使用显式的栈，而不是递归调用来完成分析
  - 类似模拟LL文法对应的下推自动机(PDA)
- **LL(1)分析的递归实现**
  - 递归下降分析: 非终结符对应子过程

Example 1: LL(1) 的递归下降实现

S -> if E then S else S	L -> end
S -> begin S L	L -> ; S L
S -> print E	E -> num = num

- **Step 1: Represent the token**

```
enum token {IF, THEN, ELSE, BEGIN, END, PRINT, SEMI,
NUM, EQ};
```

Example 1: LL(1) 的递归下降实现

S -> if E then S else S	L -> end
S -> begin S L	L -> ; S L
S -> print E	E -> num = num

- **Step 2: build infrastructure for reading tokens from lexer**

```
// call lexer
extern enum token getToken(void);

// store the next token
enum token tok;
void advance() {tok=getToken();}
// consume the next token and get the new one
void eat(enum token t) {if (tok==t) advance(); else
error();}
```

Example 1: LL(1) 的递归下降实现

S -> if E then S else S
S -> begin S L
S -> print E

L -> end
L -> ; S L
E -> num = num

- **Step 3: build a function for each non-terminal**

```
void S(void) {
  switch(tok) {
    case IF: eat(IF); E(); eat(THEN); S(); eat(ELSE); S(); break;
    case BEGIN: eat(BEGIN); S(); L(); break;
    case PRINT: eat(PRINT); E(); break;
    default: error(); }
}
void L(void) {
  switch(tok) {
    case END: eat(END); break;
    case SEMI: eat(SEMI); S(); L(); break;
    default: error(); }
}
void E(void) { eat(NUM); eat(EQ); eat(NUM); }
```

似乎不需要First/Follow?!

Example 1: LL(1) 的递归下降实现

S -> if E then S else S	L -> end
S -> begin S L	L -> ; S L
S -> print E	E -> num = num

似乎不需要First/Follow?!

- **This grammar is very very special!**
- For each non-terminal, the first symbols Y1 in the right-hand sides of its productions are different terminals
  - S: {if, begin, print}
  - L: {end, ;}
  - E: {num}

Example 2: LL(1) 的递归下降实现(另一个语法)

S -> E$	T → T * F	F -> id
E -> E + T	T → T / F	F -> num
E -> E – T	T → F	F -> (E)
E -> T		

- **How to build a function for each non-terminal?**

```
void S(void) { E(); eat(EOF); }
void E(void) {
  switch (tok) {
    case ?: E(); eat(PLUS); T(); break;
    case ?: E(); eat(MINUS); T(); break;
    case ?: T(); break;
    default: error();
  }
}
```

lookahead token

If tok == num, which production to choose?

此时需要预测分析表辅助决策

3. 文法改造

- 提左公因子
- 消除左递归

LL(1)文法的重要性质

- **LL(1)文法有一些明显的性质**
  - LL(1)文法是无二义的
  - LL(1)文法是无左公因子的
  - LL(1)文法是无左递归的

除了利用LL(1)的定义外，有时可以利用这些性质判定某些文法不是LL(1)的

LL(1)文法要求：无左公因子

- **有左公因子的(left-factored)文法**

$$P \rightarrow \alpha\beta \mid \alpha\gamma$$

- **问题: 同一非终结符的多个候选式存在共同前缀，可能导致回溯**
  - 很明显，违背了First相关的原则

思路: 限制文法或进行文法变换

文法变换: 提左公因子

- **有左公因子的(left-factored)文法**

$$P \rightarrow \alpha\beta \mid \alpha\gamma$$

- **提左公因子(left factoring)**
  - 对形如 $P \rightarrow \alpha\beta \mid \alpha\gamma$ 的一对产生式，用如下产生式替换

$$P \rightarrow \alpha Q$$
$$Q \rightarrow \beta \mid \gamma$$

Q为新增加的未出现过的非终结符

通过改写产生式来推迟决定， 等读入了足够多的输入，获得足够信息后再做选择

3. 文法改造

- 提左公因子
- 消除左递归

LL(1)文法要求: 无左递归

- **左递归(left-recursive)文法**
  - 如果一个文法中有非终结符号A使得$A \Rightarrow^{+} A\alpha$，那么这 个文法就是左递归的
  - $S \rightarrow Sa \mid b$ **(直接/立即左递归)**

- **问题: 递归下降分析可能进入无限循环**
  - 如考虑串**baaaaa**
  - 最左推导:$S \Rightarrow Sa \Rightarrow Saa \Rightarrow Saaa \Rightarrow Saaaa \ldots$

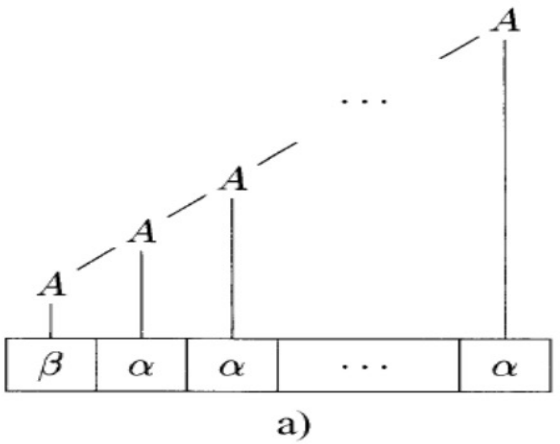
思路: 限制文法或进行文法变换

文法变换: 消除直接左递归

$A \to A\alpha \mid \beta$ ，其中$\alpha \neq \varepsilon$,α ，β 不以A开头

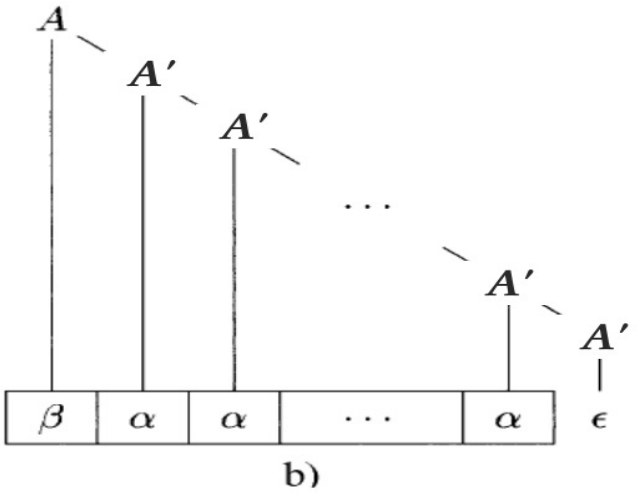
$$A \to \beta A'$$
$$A' \to \alpha A' \mid \varepsilon$$

把左递归转成了右递归



a)

- 观察：由A生成的串以某个β开头， 然后跟上零个或多个α



b)

通用的左递归消除方法(详见龙书)

4. 错误恢复(Error Recovery)

Why Error Recovery Matters

- **编译器的错误处理**
  - 词法错误，如标识符、关键字或算符的拼写错
  - 语法错误，如算术表达式的括号不配对
  - 语义错误，如算符作用于不相容的运算对象
  - …
- **错误处理的基本目标**
  1. 清楚而准确地报告错误的出现，并尽量少伪错误
  2. 迅速地从错误中恢复过来，以便诊断后面的错误
  3. 不应该使正确程序的处理速度降低太多

Example

❌ Bad Compiler UX

$ cc foo.c
error: expected ')' on line 4
1 error. Fix and recompile.

User must fix → recompile → discover next error → repeat…

✅ Good Compiler UX

$ cc foo.c
error: expected ')' on line 4
error: expected ';' on line 6
error: undeclared 'z' on line 9
3 errors

User sees all errors at once → faster edit-compile cycle.

The compiler found **both** errors in a single pass. It didn't stop after the first one — it *recovered* and continued parsing. How?

Where Errors Arise in Predictive Parsing

- Recall that a predictive parser uses a table M[A, t] indexed by nonterminal A and lookahead token t.
  - An **error** is detected when the parser looks up M[A, t] and finds a **blank entry**.

id	num	+	*	(	)	$	
E	E→T E'	E→T E'	error	error	E→T E'	error	error
E'	error	error	E'→+ T E'	error	error	E'→ε	E'→ε
T	T→F T'	T→F T'	error	error	T→F T'	error	error
T'	error	error	T'→ε	T'→*F T'	error	T'→ε	T'→ε
F	F→id	F→num	error	error	F→(E)	error	error

Error Recovery Strategies — Overview

1. Raise & Quit

Throw an exception at the first error and stop. No recovery at all.

Pros: Easy to implement

Cons: Only one error per compile; terrible UX

2. Insert Tokens

Pretend the expected token is present and continue parsing normally.

Pros: Simple logic

Cons: May cause an *infinite loop*

3. Delete Tokens

Skip input tokens until a token in the FOLLOW set is found.

Pros: Always terminates (EOF is reached)

Cons: May skip valid code

Strategy I: Raise & Quit

- When the parser encounters a blank table entry, it immediately **stops**:

Not robust and user-friendly!

```
void T( ) {
  switch (tok) {
    case ID:
    case NUM:
    case LPAREN: F(); Tprime(); break;
    default: error!
  }
}
```

  - Reports **only one error** per compilation — user must fix and recompile repeatedly.
  - Not robust: a single typo halts all further analysis.

Strategy II: Insert Tokens

- **Inserting**

When the parser expects token ***t*** but sees something else, *pretend* ***t*** was there. Print an error message, but **don't consume** any input

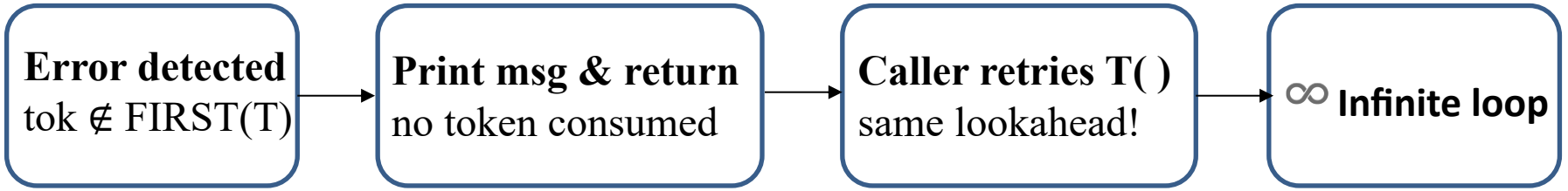
```
void T( ) {
  switch (tok) {
    case ID:
    case NUM:
    case LPAREN: F( ); Tprime( ); break;
    default:
        print("expected id, num, or left-paren");
       // don't consume input — just return  (pretend we saw a valid T)
  }
```

🚨 **Danger: Infinite Loops!**

Strategy II: Insert Tokens

- 🚨 **Danger: Infinite Loops!**
- If the parser "inserts" a token and returns, but the **caller** loops back and tries to parse the same nonterminal with the **same** lookahead token, the parser will:

1. See the same unexpected token
2. "Insert" again
3. Return to the caller again
4. Loop forever — **no input is ever consumed!**



Strategy III: Delete (Skip) Tokens

- **Deleting** ("panic-mode ")

  Idea: When an error is detected, **skip input tokens** until we find one that belongs to the **FOLLOW set** of the current nonterminal. Then return, letting the caller proceed.

```
int Tprime_follow [ ] = {PLUS, RPAREN, EOF};
void Tprime( ) {
  switch (tok) {
    case PLUS: break;
    case TIMES:
       eat(TIMES); F(); Tprime(); break;
    case RPAREN: break;
    case EOF: break;
    default:
       print("expected +, *, right-paren, or end-of-file");
       skipto(Tprime_follow);
  }
}
```

```
void skipto(int follow[]) {
   while (!member(tok, follow) &&
tok != EOF) {
      tok = nextToken(); // discard
unexpected tokens
   }
}
```

Strategy III: Delete (Skip) Tokens

- **Deleting**: skipping tokens util a token in the FOLLOW set is reached.
- Why skipto uses the Follow set?
  - Error in A: parser loses track inside A.
  - FOLLOW(A) gives a safe synchronization point.
  - Tokens in FOLLOW(A) can legally come after A.
  - Reaching FOLLOW(A) means: stop A and continue above.
- Safer than the inserting strategy:

Deleting tokens is safer, because the loop must eventually terminate when EOF is reached

Comparison: Insert vs. Delete

🔧 Token Insertion

Aspect	Detail
Mechanism	Pretend expected token exists
Input consumed	**None**
Termination	**Not guaranteed**
Cascading errors	Fewer (structure preserved)
Risk	**Infinite loop**

🗑️ Token Deletion

Mechanism	**Skip tokens to FOLLOW set**
Input consumed	**≥ 1 token**
Termination	**Guaranteed** (EOF reached)
Cascading errors	More (skipped code ≈ lost context)
Risk	**Over-skipping**

工程上可能会组合使用不同策略

Summary

- **Recursive descent parser for LL(k) grammars by:**
  - Computing nullable, first and follow sets
  - Constructing a parse table from the sets
  - Checking for duplicate entries, which indicates failure
  - Creating an C program from the parse table

- **If parser construction fails, we can**
  - Rewrite the grammar (left factoring, eliminating left recursion, etc.) and try again
  - Try to build a parser using some other method

Thank you all for your attention

一些补充例子

例 1: LL(1) 的递归下降实现

$$S \rightarrow E\,S'$$
$$S' \rightarrow \varepsilon \mid +S$$
$$E \rightarrow \mathbf{num} \mid (\,S\,)$$

- Consider a simple grammar

	num	+	(	)	$
S	$\rightarrow E\,S'$		$\rightarrow E\,S'$		
S'		$\rightarrow +S$		$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
E	$\rightarrow$ **num**		$\rightarrow (\,S\,)$		

What will be the functions for S, S', and E?

Call Tree = Parse Tree!

$$S \rightarrow E\,S'$$
$$S' \rightarrow \varepsilon \mid +\,S$$
$$E \rightarrow \textbf{num} \mid (\,S\,)$$

Consider (1 + 2 + (3 + 4)) + 5